

Stat 153/248 - Homework 5 - YOUR NAME HERE {-}

Student ID: {-}

Collaborated with: {-}

Due May 7 at 11:59pm. Grading will be completed within 14 days of the late deadline (remember you have 120 late hours you can use across the semester).

Instructions: Please complete the homework by filling out this Jupyter notebook and exporting the final file as a PDF or .html file. (Go to File menu -> Save and Export Notebook as -> choose PDF or html, save, and upload this file as your submission.

You should ideally write out your solutions as markdown / LaTeX within this notebook. If you do decide to include any handwritten notes, these must be incorporated into one PDF (including all your code, solutions, etc) and each problem must be clearly labeled with the question number. Everything must be submitted as one single PDF. Points will be deducted if questions are not clearly labeled and formatting guidelines are not followed.

Remember, if you collaborated with anyone, you should list their names on this document, but your answers must be your own (unique, not a copy of/identical to a friend's). This homework will be graded for completion, so while you may use external tools to help you in completing it, it is recommended that you try to figure out the solutions and understand them yourself.

Instructions:

Below is some helpful information for you as you solve these homework problems. "Part 0" will also cover some important intro material to help supplement what you saw in Lecture.

These models don't work well in the datahub, so you'll want to [install pytorch on your own computer](#).

Convolution output shape formula

For a 1D convolution with input length L , kernel size k , padding p , and stride s , the output length is

$$L_{\text{out}} = \left\lfloor \frac{L + 2p - k}{s} \right\rfloor + 1.$$

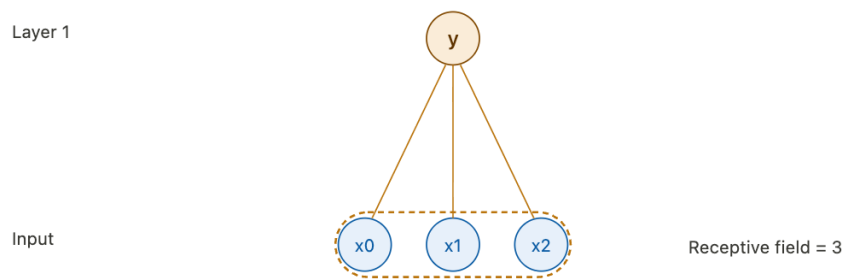
You may apply this over multiple independent batches and using different numbers of filters (with some kernel size).

For 2D, the same formula applies independently to each spatial dimension.

Receptive field

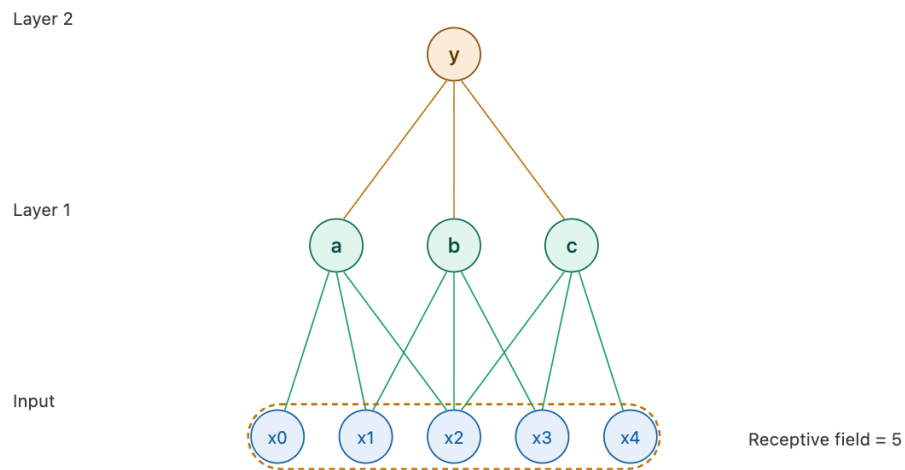
The **receptive field** of an output unit is the number of input positions that unit depends on. It is a property of the architecture and does not change with input length.

For a single conv layer with kernel size k , each output unit sees k consecutive input positions, so the receptive field is k .



When you stack conv layers, the receptive field grows. Let's consider two 1D conv layers, each with kernel size $k = 3$:

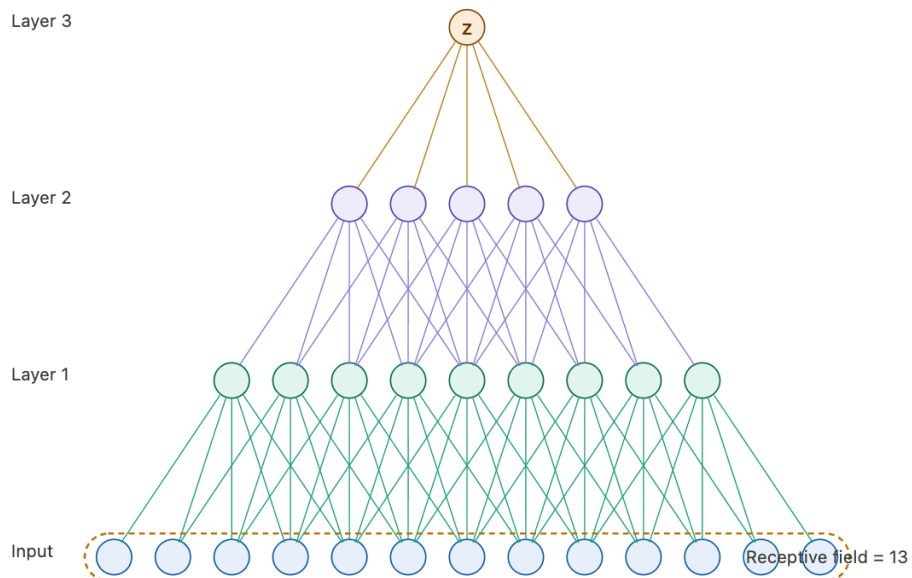
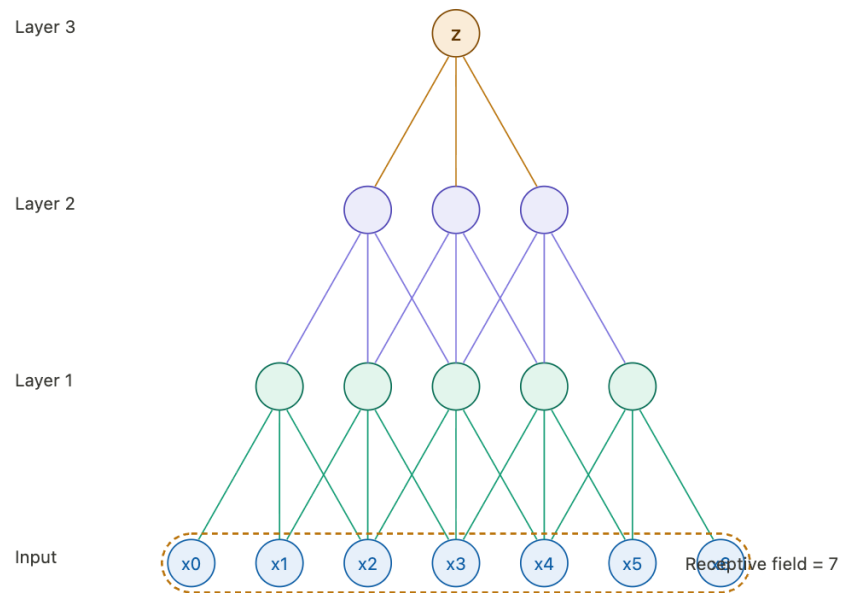
- One unit in layer 2 sees 3 consecutive units in layer 1.
- Each of those 3 layer-1 units sees 3 consecutive units in the input.
- The leftmost layer-1 unit covers input positions 0, 1, 2. The rightmost covers 2, 3, 4.
- So a single layer-2 unit depends on input positions 0 through 4 — a receptive field of **5**.



In general, for n stacked conv layers with kernel size k (stride 1):

$$\text{RF}_n = n(k - 1) + 1.$$

Two 3×3 layers $\rightarrow \text{RF} = 5$. Three 3×3 layers $\rightarrow \text{RF} = 7$. This is why deep networks built from small kernels can still cover large input regions: receptive field grows linearly with depth. On the left, we have 3 1D conv layers with kernel size $k = 3$, on the right, 3 1D conv layers with kernel size $k = 5$:



Note that **receptive field is different from output length**. Output length tells you how many positions remain after the convolutions; receptive field tells you, for one of those output positions, how many input positions contributed to it. Both involve $(k - 1)$ terms but they answer different questions.

```
In [ ]: import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
torch.manual_seed(0)
np.random.seed(0)
```

Part 0: Warm-up

Before we start on the main problems, we'll work through this short warm-up to make sure you are comfortable with PyTorch's shape conventions. This helps to set you up for Question 1.

Shape conventions for Conv1d

A 1D convolutional layer in PyTorch expects input tensors of shape

(batch, channels, length).

- **batch**: how many independent signals you are processing in parallel (e.g., 32 EEG trials).
- **channels**: how many parallel values exist at each timepoint (e.g., 1 for a univariate signal, 64 for 64 electrodes, 40 for 40 frequency bins). Channels are *not* a spatial axis. Rather, the kernel looks across all of them simultaneously at each position.
- **length**: the temporal axis. This is what the kernel slides along, and this is the L in the output-length formula.

The "1D" in "1D convolution" means the kernel slides along **one axis** (length). We never slide over batch or channels.

Worked example

Suppose the input has shape (batch = 2, channels = 1, length = 10) and we apply a `Conv1d` with 4 filters, kernel size 3, no padding, stride 1. We can use the formula at the top of this notebook:

- Output length: $L_{\text{out}} = \lfloor (10 + 0 - 3) / 1 \rfloor + 1 = 8$.
- Output channels: equal to the number of filters = 4.
- Output shape: (2, 4, 8).

Each of the 4 filters has shape (in_channels = 1) $\times$ (kernel_size = 3) = 3 weights, so the layer has $4 \times 3 = 12$ weight parameters (plus 4 biases if biases are enabled).

Verify in code

Run the cell below to confirm the shape calculation, then modify it and predict before running.

```
In [ ]: # Build the layer from the worked example
layer = nn.Conv1d(in_channels=1, out_channels=4, kernel_size=3)

# Apply it to a batch of 2 univariate signals of length 10
x = torch.randn(2, 1, 10) # initialize some random data
out = layer(x) # apply convolutional layer

print("input shape: ", tuple(x.shape))      # expect (2, 1, 10)
print("output shape:", tuple(out.shape))    # expect (2, 4, 8)
print("weight shape:", tuple(layer.weight.shape)) # expect (4, 1, 3)
print("n weight params:", layer.weight.numel()) # expect 12
```

```
In [ ]: # ---- Predict-then-verify ----
# Before running the lines below, predict:
# 1. What output shape do you expect if kernel_size is 5 instead of 3?
# 2. What if you add padding=2 (with kernel_size=5)?
# 3. What if the input has 3 channels instead of 1 (so x.shape = (2, 3, 10)
#    with kernel_size=3 and out_channels=4? How many weights does the layer
#    have?
# Uncomment one block at a time after making your prediction.

# (1) kernel_size=5
# print('\nkernel_size=5, padding=0, stride=1, length=10')
# layer2 = nn.Conv1d(1, 4, kernel_size=5)
# print(layer2(torch.randn(2, 1, 10)).shape)

# (2) kernel_size=5, padding=2
# print('\nkernel_size=5, padding=2, stride=1, length=10')
# layer3 = nn.Conv1d(1, 4, kernel_size=5, padding=2)
# print(layer3(torch.randn(2, 1, 10)).shape)

# (3) in_channels=3
# print('\ninput_channels=3, kernel_size=3, padding=0, stride=1, length=10')
# layer4 = nn.Conv1d(3, 4, kernel_size=3)
# print(layer4(torch.randn(2, 3, 10)).shape)
# print("\nn weight params:", layer4.weight.numel())
```

Expected answers (check after you run the code):

1. Output shape (2, 4, 6) — length drops by $k - 1 = 4$.
2. Output shape (2, 4, 10) — "same" padding preserves length.
3. Output shape (2, 4, 8). The layer has $4 \times 3 \times 3 = 36$ weights: each filter now looks across all 3 input channels, so the per-filter weight count is $\text{in_channels} \times \text{kernel_size}$.

In the third case, note that `in_channels` multiplies into the per-filter weight count, even though the kernel only slides along the length axis. This will matter for Question 2, which uses 3 RGB input channels.

Now let's start the questions.

Part 1: Convolution mechanics

The first four questions ask you to compute output shapes and parameter counts by hand, then verify your answers in code.

Question 1 (4 points)

A 1D convolutional layer has:

- input shape `(batch, 1, 100)` or `(batch, channels, length)`. Batch is how many independent samples we process at once, channels is how many parallel signals at each point, and length is the temporal axis that the kernel slides along.
- 8 filters
- kernel size 7
- no padding
- stride 1

Compute:

(a) The output shape.

(b) The total number of *weights* in this layer (ignoring biases - which can be used if the input data and features are not centered).

(c) The total number of weights if this layer were fully connected instead (i.e., every input element connected to every hidden unit, where the number of hidden units equals the flattened output size from part (a)).

(d) The ratio of fully-connected to convolutional weights. Briefly explain in one sentence what property of convolutional layers is responsible for this ratio.

Your answer:

(a)

(b)

(c)

(d)

Question 2 (2 points)

A 2D convolutional layer takes an RGB image of shape `(batch, 3, 64, 64)` and applies 16 filters of size 5×5 with `padding=2` (same padding) and `stride=1`.

(a) What is the output shape?

(b) How many weights does this layer have (ignore biases)? Be explicit about where each factor comes from.

Your answer:

(a)

(b)

Question 3 (2 points)

Consider three padding strategies for a 1D convolution with kernel size 5 applied to an input of length 20:

- **Valid padding** ($p = 0$)
- **Same padding** ($p = 2$)
- **Full padding** ($p = 4$)

(a) Compute the output length for each (stride 1).

(b) In lecture 23 there was a brief discussion about padding choices in a *time series* context (forward vs. backward in time). Briefly describe what "causal padding" would mean and why it matters for a model that should not peek into the future.

Your answer:

(a)

(b)

Part 2: Architecture

For these questions, we will talk about the architecture of CNNs and how to calculate the number of weights in different networks.

Question 4 (5 points)

Lecture 24 argued that a fully-connected network for a 256×256 RGB image (3 color channels) with 1000 hidden units and 1000 output classes has a very large number of weights. Compute this number and show your work. Then compute the weight count if each of the 1000 hidden units has its receptive field restricted to 11×11 pixels (still using all 3 color channels). What is the reduction factor?

Your answer:

Question 5 (3 points)

State whether each of the following is true or false, and give a one-sentence justification.

- (a)** Max pooling over a filter's response map makes the network's decision approximately invariant to *where* in the input the feature appears.
- (b)** A ReLU nonlinearity can be replaced by a linear function without loss of expressiveness, as long as enough layers are stacked.
- (c)** Two stacked 3×3 convolutional layers have the same effective receptive field as one 5×5 convolutional layer. (Hint: compute the

Your answer:

(a)

(b)

(c)

Part 3: Code exercises

Use the code cells below. Answers should be kept short.

Question 6 (3 points): verify your shape calculations

Construct an `nn.Conv2d` layer matching the specification in Question 2 (input `(1, 3, 64, 64)`, 16 filters of size 5×5 , same padding). Apply it to a random input tensor using `torch.randn` for an input of shape `(1, 3, 64, 64)`. Print both the output shape and the total number of weights in the layer's `.weight` parameter. Confirm that they match your hand calculations from Question 2.

```
In [ ]: # Solution for Q6
layer = nn.Conv2d(## FILL IN##)
x = torch.randn(##FILL IN##)
out = # Apply the conv layer

# Print output shape:

# Print weight tensor shape:

# Print total number of weights:
```

Question 7 (5 points): parameter counting

Define a small 1D CNN for waveform classification with the architecture below. Then write code that prints the number of *trainable* parameters in each layer and the total.

```
Conv1d(1  -> 16, kernel_size=32, stride=4, padding=16)  ->
ReLU -> MaxPool1d(4)
Conv1d(16 -> 32, kernel_size=3, padding=1)              ->
ReLU -> MaxPool1d(4)
Conv1d(32 -> 64, kernel_size=3, padding=1)              ->
ReLU -> AdaptiveAvgPool1d(1)
Linear(64 -> 10)
```

Hint: biases count as parameters. A `Conv1d(in, out, k)` layer has `in*out*k + out` parameters total; `Linear(a, b)` has `a*b + b`.

```
In [ ]: # Solution for Q7
model = nn.Sequential(
    nn.Conv1d(##FILLIN),
    nn.ReLU(##FILLIN),
    nn.MaxPool1d(##FILLIN),
    ## ADD The OTHER LAYERS HERE
)

total = 0
for name, p in model.named_parameters():
    if p.requires_grad:
        print(f"{name:20s} {tuple(p.shape)!s:20s} {p.numel():>6d}")
        total += p.numel()
print(f"{'total':20s} {'':20s} {total:>6d}")

# Expected (by hand):
# conv1 weight: 1*16*32 = 512, bias: 16    -> 528
# conv2 weight: 16*32*3 = 1536, bias: 32   -> 1568
# conv3 weight: 32*64*3 = 6144, bias: 64   -> 6208
# fc weight: 64*10 = 640, bias: 10         -> 650
# total = 8954
```

Question 8 (2 points)

You are designing a classifier for two different tasks. For each, state whether you would use a **1D CNN on the raw waveform**, a **2D CNN on a spectrogram**, or a **non-convolutional model** (e.g., logistic regression on hand-engineered features). Justify your choice in one or two sentences referencing specific properties of convolutional models.

(a) Detecting a single, fixed, known template (e.g., a specific epileptic spike waveform) in continuous time series brain data (EEG).

(b) Classifying spoken words from raw audio, where pronunciation varies across speakers and time.

Your answer:

(a)

(b)

Question 9 (2 points)

A student trains two models on the same audio dataset with the same architecture and hyperparameters. The only difference is the train/test split:

- **Model A:** random 80/20 split of all recordings.
- **Model B:** held-out speaker where all recordings from one speaker are in the test set, none in the training set.

Model A achieves 95% test accuracy; Model B achieves 45% test accuracy.

(a) Explain the gap. What has Model A likely learned that Model B is failing to exploit?

(b) Which number better reflects how well the model will perform on a *new user* of the system, and why?

Your answer:

(a)

(b)

Part 4: Recurrent neural networks

We have not covered RNNs in lecture yet. This short section introduces the essentials and compares RNNs to the CNNs you just worked through.

Primer

A **recurrent neural network (RNN)** processes a sequence $x_1, x_2, \dots, x_T$ one step at a time, maintaining a hidden state h_t that summarizes everything seen so far and predicts $\hat{y}_t$. The simplest ("vanilla") RNN applies the same update equation at every timestep:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h), \quad \hat{y}_t = W_y h_t + b_y$$

Here $W_h \in \mathbb{R}^{H \times H}$, $W_x \in \mathbb{R}^{H \times d}$, $W_y \in \mathbb{R}^{k \times H}$, where H is the hidden size, d is the input dimension per timestep, and k is the output dimension. The nonlinearity is usually $\tanh$ or ReLU .

We fit the following parameters:

- W_x , which maps the current input x_t into the hidden space
- W_h , which maps the previous hidden state h_{t-1} forward
- b_h , which is the bias term for the hidden update
- W_y , which maps the hidden state to the output
- b_y , the bias for the output

There are three important properties:

1. **Weight sharing across time:** The same (W_h, W_x, W_y) are used at every timestep. This is analogous to a convolutional filter being applied at every spatial position.
2. **Unbounded context (in principle):** The hidden state h_t can depend on arbitrarily distant past inputs through the recurrence, unlike a CNN whose receptive field is fixed by architecture.
3. **Training is sequential:** To compute h_t you need h_{t-1} , so forward and backward passes run step-by-step along the sequence. This is called backpropagation through time (BPTT). It means RNNs are harder to parallelize than CNNs, which compute all positions in a layer at once.

Vanishing/exploding gradients. When you backpropagate through many timesteps, the gradient is multiplied by W_h repeatedly. If the eigenvalues of W_h are less than 1 in magnitude, gradients shrink to zero over long sequences and the network cannot learn long-range dependencies. If greater than 1, gradients explode. **LSTM** and **GRU** cells are architectural modifications that add gating mechanisms to control information flow; they preserve gradients over much longer timescales and are what people actually use in practice. For this homework, "LSTM" and "GRU" can be treated as black-box drop-in replacements for the vanilla RNN that handle long sequences better.

Here are some rough rules of thumb for time series tasks:

Property	CNN (1D)	RNN (LSTM/GRU)
Receptive field	fixed by architecture	unbounded (in principle)
Parallelism across time	yes	no
Training speed	fast	slow
Good for	local patterns, fixed-scale features	long-range dependencies, variable-length sequences

Modern practice often combines the two, using a CNN frontend for efficient local feature extraction, followed by an RNN (or transformer) for long-range integration. This is the architecture behind models like wav2vec 2.0 mentioned in lecture 23.

Question 10 (2 points): parameter counting for an RNN

Consider a vanilla RNN with input dimension $d = 10$, hidden size $H = 64$, and output dimension $k = 3$ (the output is produced at every timestep).

(a) Compute the total number of trainable parameters in the RNN (include biases: one bias vector for the hidden state update, one for the output). Show each term.

(b) Compare to a 1D CNN with the same input dimension ($d = 10$ channels), 64 output channels, and kernel size 3 (no bias, for simplicity). How many parameters does the CNN have? What does the comparison tell you?

Your answer:

(a)

(b)

Question 11 (3 points): When to use a RNN vs. a CNN?

For each of the following time series tasks, state whether you would start with a **1D CNN**, an **RNN (LSTM/GRU)**, or a **hybrid (CNN frontend + RNN)**. Justify each answer in one or two sentences citing a specific property from the primer.

(a) Forecasting daily electricity demand 24 hours ahead, where the relevant patterns include weekly cycles (7 days) and annual cycles (365 days). You have one data point per hour.

(b) Detecting epilepsy related activity (also called interictal epileptiform discharges (IEDs)) in continuous intracranial EEG recordings from the brain. IEDs are stereotyped short (~100 ms) waveforms and the model should output a label per time window.

(c) Classifying full sentences of speech (variable length, several seconds, where word-level meaning depends on long-range context across the sentence) from audio waveforms.

Your answer:

(a)

(b)

(c)

In []: